Fire Sentinel — Prototype 2

AI-assisted multi-sensor fire detection and alerting system

Project report & viva preparation guide

ESP32-S3 camera board · gas / UV / flame sensors · on-device CNN + YOLO
Flask + SQLite backend · Telegram / Email alerts · React dashboard

State of the project: firmware v2.3 (29 Oct 2026)

Contents

1. Problem statement and objectives
2. System architecture
3. Hardware and wiring
4. Firmware (ESP32-S3)
5. Machine learning: CNN, YOLO, sensor model
6. Backend (Flask + SQLite)
7. Dashboard (React)
8. End-to-end scenarios
9. Testing and verification
10. Limitations and future work
11. Viva questions and answers
12. Quick-reference numbers

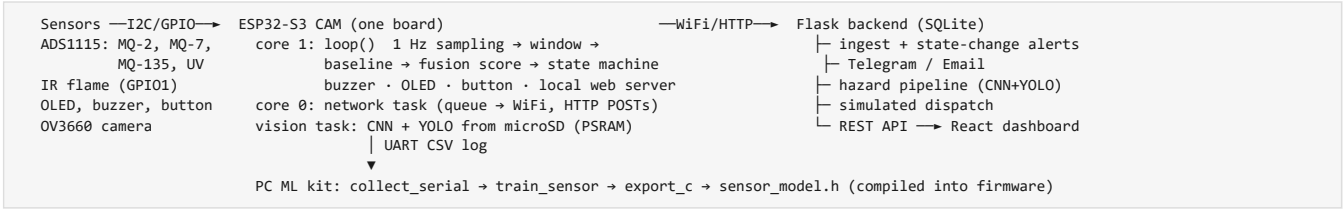
1. Problem statement and objectives

Conventional smoke detectors use a single sensor and a fixed threshold. They give false alarms (burnt toast, steam, a candle), cannot tell a controlled flame from a dangerous fire, and do not notify anyone who is away from home.

Fire Sentinel is a low-cost IoT device that:

- combines several independent signals (smoke/combustible gas, CO, VOC, UV light, IR flame, and optionally temperature) into one transparent **0–100 risk score**;
- runs a **four-stage alarm state machine** (NORMAL → PRECURSOR → CONFIRMING → FIRE_CONFIRMED) that needs evidence to *persist* before escalating, which rejects transient false alarms;
- takes a **camera snapshot** as evidence and classifies it **on the board** with a quantized CNN (NO_FIRE / CONTROLLED_FLAME / HAZARDOUS_FIRE) and a YOLO flame detector;
- works **offline**: the alarm decision, buzzer and display run locally even with no WiFi;
- sends **Telegram and Email** alerts with the photo, logs everything to a database, shows it on a web dashboard, and runs a **simulated fire-department dispatch** countdown that the owner can cancel.

2. System architecture



Layer	Responsibility
ESP32-S3 device	Sense, decide the alarm state locally, sound the buzzer, show status on the OLED, classify snapshots, upload data and photos.
Backend (PC)	Store history, send alerts once per state change, analyse photos, run the dispatch countdown, serve the API.
Dashboard	Live gauges, camera, snapshot gallery, detection timeline, device and room setup.
ML kit (PC, offline)	Train and evaluate the CNN, YOLO and sensor models; quantize/export them for the device.

Key design principle: the safety-critical decision is made on the device, by rules that can be explained. ML adds evidence (it can *raise* the risk, never lower it), and the cloud/backend is for notification and record-keeping only.

3. Hardware and wiring

3.1 Components (current build, v2.3)

Module	Role
Freenove ESP32-S3-WROOM CAM (N16R8, OV3660)	Main board: dual-core 240 MHz, WiFi, camera, 8 MB PSRAM, 16 MB flash, microSD slot.
ADS1115 (I2C 0x48)	16-bit external ADC. Needed because the camera occupies most ADC1 pins and ADC2 cannot be used while WiFi is on.
MQ-2 / MQ-7 / MQ-135	Smoke & combustible gas / carbon monoxide / air quality-VOC. Read on ADS A0/A1/A2 through 10k/10k dividers.
GUVA-S12D UV sensor	Flames emit UV. ADS A3 (direct).
B35 IR flame sensor	Digital flame output (LM393 comparator) on GPIO1, active LOW.
SSD1306 OLED 128×64 (0x3C)	Local status screens: STATUS, SENSORS, VISION, NETWORK.
Buzzer (GPIO42)	Distinct pattern per alarm state (LEDC PWM tones).
Info button (GPIO2)	Short press: next screen. Long press 1.5 s: silence + acknowledge (cancels dispatch).
microSD card	Holds the two quantized models (`fire_cnn_int8.tflite`, `fire_yolo_int8_160.tflite`).
Power	5 V / 2 A USB-C adapter. MQ heaters (~450 mA total) run from the board's 5V pin.

> **Design history:** v2.2 had a LiPo battery + TP4056 charger + boost converter + INA219 power monitor, plus three status LEDs and a second "diffuse" button. In v2.3 these were removed: power is USB-C, the single info button does both jobs. The BME280 temperature/humidity/pressure sensor is supported in code but **not fitted** in the current build (`BME280_FITTED 0`), so the fusion uses flame, gas and UV.

3.2 Pin map

GPIO	Use
1	IR flame DO (active LOW)
2	Info button (internal pull-up, other leg to GND)
20 / 21	I2C SDA / SCL (ADS1115, OLED, optional BME280)
42	Buzzer
38 / 39 / 40	Onboard microSD (SD_MMC, 1-bit)
4–13, 15–18	Camera ribbon (fixed by the board)
43 / 44	UART0: flashing, logs, training CSV
26–37 · 0, 3, 45, 46	Reserved for PSRAM/flash · strapping pins — deliberately avoided

3.3 Wiring decisions worth explaining

- **Voltage dividers on MQ outputs:** MQ modules run at 5 V, their output can reach 5 V; the ADS1115 is powered at 3.3 V. A 10k/10k divider halves it to ≤ 2.5 V. Firmware multiplies by `MQ_DIVIDER_RATIO = 2.0`.
- **I2C kept at 3.3 V;** only the MQ heaters use 5 V. The flame sensor is powered at 3.3 V so its digital output is a safe logic level for the ESP32.
- **GPIO20 is also native-USB D+.** It works as SDA because "USB CDC On Boot" is disabled and the board is flashed via the UART USB port. The native USB port must stay unplugged.
- **Common ground** for all modules; I2C devices have different addresses so they share one bus.

4. Firmware (ESP32-S3, Arduino core 3.3.12)

4.1 Execution model

- `loop()` **on core 1** — everything time-critical and local. It never waits on the network.
- **Network task on core 0** — owns WiFi and HTTP. `loop()` pushes small jobs into a FreeRTOS queue (length 8). If WiFi or the backend fails, sensing and the alarm keep working; failed uploads are dropped; after 3 failures the task backs off 5 s.
- **Vision task** — runs the CNN and YOLO on snapshots so inference never blocks the alarm loop.

4.2 One second of the main loop (fixed 1 Hz)

1. `sensors_read()` — ADS1115 (4 channels) and BME280 if fitted; checks every I2C device still answers.
2. Push the sample into a 30-sample ring buffer (reset it if the read was invalid).
3. `window_compute()` — mean, standard deviation and least-squares slope per channel.
4. Run the sensor ML model (advisory) if one is compiled in.
5. `fusion_compute()` — rule-based 0–100 risk score; a fresh HAZARDOUS camera verdict (≥ 80 %) may raise it.
6. `state_machine_update()` — new state; decide whether to take a snapshot.
7. If NORMAL → update the baseline. Update buzzer, print CSV line, queue uploads.

Why a fixed 1 Hz? Features such as slope and standard deviation only mean the same thing during training and on the device if the sample rate never changes. The alarm state changes only the *upload* rate.

4.3 Sensors and adaptive baseline (`sensors.cpp`)

- **Flame debounce:** pin polled every 100 ms into a 10-slot history; flame = ≥ 4 of the last 10 polls active. A flickering real flame counts, a single glitch does not.
- **Raw ADC counts** (0.125 mV/count) are used for decisions and ML. The `*_ppm` values are placeholder estimates for display only — MQ sensors are not calibrated gas meters.
- **Baseline calibration:** MQ heaters warm up for 3 min; then 60 quiet samples set the gas/UV baseline (ready ≈ 4 min after power-on). Learning happens only in NORMAL.
- **Protection against "learning" a slow fire as normal:** learning freezes when any MQ is > 25 % above baseline; baseline is clamped to 0.5–1.6× of its first value; slow EMA (~5 min).
- Baseline saved to flash (NVS) every 30 min and restored at boot; hold the button 3 s at power-on to wipe it (e.g. after moving rooms).
- **Fault handling:** a disconnected I2C device is flagged (`sensor_fault`, OLED shows FLT), retried every 10 s and recovers without reboot.

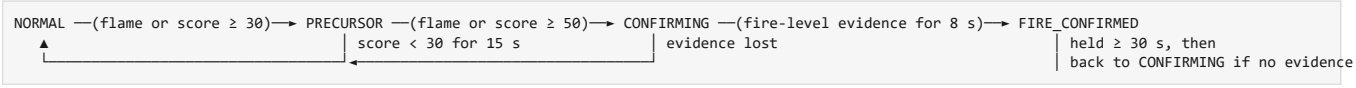
4.4 Fusion risk score (`fusion.cpp`)

Component	Max points	How it is earned
Flame	35	Debounced IR flame detected
Temperature (if BME280 fitted)	25	Best of: +5 °C above baseline, +6 °C/min rate of rise, absolute ≥ 57 °C
Gas	30	Worst MQ deviation vs baseline (+150 % = full), after warm-up
Multi-gas agreement	10	≥ 2 MQ sensors above +40 %
UV	10	UV above baseline (~0.3 V = full)

Total is clamped to 100. Thresholds: **PRECURSOR 30 · CONFIRMING 50 · FIRE 65**. A temperature ≥ 57 °C forces at least CONFIRMING (like a fixed-temperature heat detector). The weights are hand-set placeholders, to be tuned on real recorded data.

Why rule-based? It is transparent, explainable in a viva or to a safety inspector, and needs no training data — which we do not yet have for real fires.

4.5 Alarm state machine (`state_machine.cpp`)



- **Fire-level evidence** = score ≥ 65 on its own, **or** flame plus corroboration (score ≥ 50). A flame alone (sunlight, lamp, candle) stops at CONFIRMING.
- **Hysteresis:** 8 s persistence to escalate, 15 s cooldown to de-escalate, 30 s minimum FIRE hold — this rejects a lighter flick.
- A snapshot is taken on entering PRECURSOR, CONFIRMING and FIRE_CONFIRMED.

4.6 Buzzer, button and OLED

State	Buzzer	Backend alert
NORMAL	silent	—
PRECURSOR	short beep every 6 s	Telegram (info)
CONFIRMING	double beep every 1.5 s	Telegram + Email (warning)
FIRE_CONFIRMED	continuous tone	Telegram + Email (critical) + dispatch countdown

Silencing affects only the buzzer, never the state or alerts. It lasts 5 min (60 s in FIRE_CONFIRMED) and re-arms when severity increases. The OLED sleeps after 30 s in NORMAL and wakes/pins to STATUS on any alarm.

4.7 On-device vision (`vision.cpp`, new in v2.3)

- At boot the microSD is mounted and both `.tflite` models (CNN 627 KB, YOLO 2.9 MB) are loaded into PSRAM and run with **TensorFlow Lite Micro**.
- Pre-processing matches training exactly: JPEG (SVGA) decoded at 1/4 scale → CNN: area-resize to 96×96, int8 = pixel − 128 · YOLO: letterbox to 160×160 (pad 114).
- YOLO post-processing: confidence 0.25, NMS IoU 0.45, max 5 boxes.
- A custom int8 FULLY_CONNECTED kernel was written because the built-in TFLM kernel aborts on YOLO's attention block (non-zero weight zero-point).
- Crash safety: a marker in RTC memory records which model was running; if the board resets during inference, that model disables itself instead of boot-looping the fire alarm.
- No SD card / missing file → vision disabled, alarm still works. Set `VISION_AFFECTS_SCORE 0` to make vision advisory only.

4.8 Networking

- Device id `fs-<MAC>`; self-registers with the backend (id, IP, port) at WiFi connect and every 60 s — no reflash when IP or room changes.
- Uploads to `POST /api/sensor-data`: every 30 s heartbeat in NORMAL, every 1 s while elevated, immediately on state change. Payload includes state, score, raw sensor counts, fusion terms, window stats, ML results, `ack_seq`.
- Local web server: `GET /capture` (JPEG on demand, used by the dashboard live view via the backend proxy) and `GET /status`.

4.9 Firmware files

File	Purpose
`esp32s3_cam_main.ino`	setup(), 1 Hz loop, web handlers, CSV log
`config.h` / `secrets.h`	All pins, thresholds, timings · WiFi credentials
`sensors`, `window`, `fusion`, `state_machine`	Reading + baseline · 30 s window statistics · risk score · alarm logic
`sensor_ml`, `sensor_model.h`	On-device RandomForest + IsolationForest (stub until real data)
`vision`, `camera`	SD + TFLite Micro CNN/YOLO · camera capture with mutex
`buzzer`, `buttons`, `display`	Alert patterns · debounced button · OLED screens
`network_client`, `device_register`, `i2c_util.h`	Network task and queue · registration · I2C ping helper

5. Machine learning

Model	Input	Output	Runs on	Status
CNN — MobileNetV2 α=0.35, 96 px, INT8	Camera frame	NO_FIRE / CONTROLLED_FLAME / HAZARDOUS_FIRE	ESP32 (microSD)	Trained, quantized, deployed
YOLO11n, 1 class "fire"	Camera frame	Flame bounding boxes → size and growth	ESP32 (INT8, 160 px) and backend	Trained; secondary signal
RandomForest + IsolationForest	30 s window of 16 sensor features	Class + "not normal" flag	ESP32 (compiled into flash)	Pipeline built; needs real data

5.1 CNN snapshot classifier

Dataset preparation (`tools/dataset_split.py`):

Step	Count
Images found	21,028
Duplicates removed	1,322
Cross-class label conflicts dropped	79
Kept	19,632
Train / Val / Test	13,743 / 2,945 / 2,944
Test per class (no_fire / controlled / hazardous)	1,263 / 1,186 / 495

The split is **grouped** (similar images never span train and test), so the test score is not inflated by leakage.

Training (`train_cnn.py`): transfer learning from ImageNet-pretrained MobileNetV2 (width α = 0.35, 96×96 input), class weights to counter imbalance (hazardous weight 2.97 vs ~0.8 for others), early stopping after 23 epochs, about 7 minutes.

Quantization (`quantize_int8.py`): full-integer post-training quantization using 300 representative images. Input/output are int8.

Model	Size	Accuracy	Macro F1	Hazardous recall	False hazard alarm
Keras FP32	4,543 KB	97.76 %	0.970	91.52 %	0.78 %
TFLite FP32	1,573 KB	97.76 %	0.970	91.52 %	0.78 %
TFLite INT8 (deployed)	**612 KB**	**96.98 %**	**0.961**	**91.92 %**	**1.39 %**

INT8 is **2.57× smaller** than FP32 TFLite for a 0.78-point accuracy drop; hazardous recall is unchanged. Host latency ≈ 0.44 ms (PC, not the ESP32).

INT8 confusion matrix (test set, 2,944 images) — rows = true class, columns = predicted:

	no_fire	controlled_flame	hazardous_fire
no_fire	1226	5	32
controlled_flame	10	1174	2
hazardous_fire	40	0	455

The main error is hazardous fire predicted as no_fire (40 of 495 = 8.1 %), typically heavy smoke with little visible flame. That is why the camera never *lowers* the risk: sensors still drive the alarm.

Threshold trade-off: instead of arg-max, an alarm threshold on P(hazardous) can be chosen. For INT8: threshold 0.20 → recall 96.4 %, false alarm 3.1 %; threshold 0.50 → 92.3 % / 1.5 %; threshold 0.90 → 78.6 % / 0.3 %. For a safety system, higher recall is usually preferred.

5.2 YOLO11n flame detector

- Single class "fire", built from Roboflow datasets (`prepare_yolo_dataset.py`), trained on Google Colab GPU: 50 epochs, 640 px, batch 16, from `yolo11n.pt`.
- Final validation: precision 0.79, recall 0.61, mAP@50 0.69, mAP@50-95 0.34.
- Exported to INT8 at 160×160 for the ESP32 (2.9 MB).
- Used as a **secondary** signal: flame area ≥ 15 % of frame, or area growing ≥ 1.5× within 3 min, marks a snapshot HAZARDOUS. "No boxes found" is never treated as proof of no fire, because box recall is limited.

> **Be ready for this:** `yolo_test_metrics.json` shows much lower test numbers (mAP@50 0.035) because that evaluation ran at 320 px while the model was trained at 640 px. The on-board model runs at 160 px, so its accuracy is likely below the 640-px validation figure. This is a reason YOLO is advisory, and re-evaluating at the deployed resolution is listed as future work.

5.3 Sensor model (three stages)

1. **Baseline calibration (not ML)** on the device — all features are relative to the room's own baseline, so the model transfers across rooms, sensor units and days.
2. **RandomForest classifier** (40 trees, depth ≤ 8) on 16 window features → NORMAL / CONTROLLED_FLAME / HAZARDOUS_FIRE.
3. **IsolationForest anomaly detector** trained on NORMAL windows only → flags "not normal" even for fire types never recorded. Threshold = 99th percentile of normal scores.

16 features: temperature (deviation, std, slope/min); for each MQ-2, MQ-7, MQ-135 (relative mean, relative std, relative slope/min); UV (deviation, std, slope); flame duty.

Pipeline: `collect_serial.py` records labelled sessions from the serial CSV → `train_sensor.py` (leave-one-session-out evaluation, because adjacent windows overlap 29/30 and a random split would leak) → `export_c.py` converts the trees to C arrays in `sensor_model.h` → compiled into firmware flash.

Status: pipeline tested on synthetic data only; no real fire sessions recorded yet, so the firmware ships with a stub model and the result is advisory (`SENSOR_ML_AFFECTS_SCORE 0`). When enabled, it may only *raise* the score.

6. Backend (Flask + SQLite)

6.1 Sensor reading flow

```
ESP32 ──POST /api/sensor-data──► sensor_routes.py ──► risk_score_service.process_sensor_reading()
   1. create device row if unknown         4. update device last_state / last_seen
   2. gas-source hint (gas_pattern)        5. state changed? → event + Telegram/Email (with newest snapshot)
   3. INSERT sensor_readings               6. dispatch_service.on_reading() → simulated countdown
```

Alerts fire **once per state change**, not per upload, so the 1 Hz elevated rate does not spam the user.

6.2 Snapshot → hazard pipeline

`POST /api/camera/snapshot` → CNN classification (uses the board's result if the backend model is absent) → YOLO flame boxes → `hazard_service.assess` → level **HAZARDOUS / CONTROLLED / NONE / UNKNOWN**. Escalation is one-way: any single hazardous signal wins.

6.3 Simulated fire-department dispatch

- FIRE_CONFIRMED → owner gets a Telegram message and a **30 s countdown** starts.
- Cancel by long-pressing the device button (`ack_seq` increases) or `POST /api/dispatch/cancel`.
- No cancel → a message marked **[SIMULATION — no real call was placed]** with room, risk, camera verdict and photo goes to a "Fire Dept (SIM)" Telegram chat.
- Still on fire 2 min after a cancel → countdown restarts. A background timer fires it even if the device went offline mid-fire.
- Deliberately never calls a real number: unverified automated emergency calls are a liability and a false-alarm risk.

6.4 REST API (main routes)

Route	Purpose
`POST /api/sensor-data`, `GET …/latest`, `GET …/history`	Ingest, live tiles, trend charts
`POST /api/device/register`, `GET /api/devices`, `PUT /api/devices/<id>/room`	Self-registration, device list, room assignment, IP override
`POST /api/camera/snapshot`, `GET /api/camera/capture-proxy`, `GET /api/camera/snapshots`	Upload, live view, gallery
`GET /api/events`, `GET /api/alerts/history`, `POST /api/alerts/test`	Timeline, alert log, test alert
`/api/rooms`, `/api/appliances`, `/api/contacts`	Room profiles, appliances, emergency contacts
`GET /api/dispatch/status`, `POST /api/dispatch/cancel`, `GET /api/dispatch/events`	Simulated dispatch
`GET /api/model-info`	CNN model manifest

6.5 Database tables

`rooms`, `appliances`, `emergency_contacts`, `devices`, `sensor_readings` (including raw counts, `window_json`, `ml_json`), `events` (timeline), `snapshots` (CNN class, YOLO boxes, hazard level), `alerts` (each Telegram/Email attempt), `dispatch_events`. The schema migrates itself at start-up.

6.6 Services

`risk_score_service` (ingest brain), `telegram_service` (Bot API, text + photo), `email_service` (Gmail SMTP with attachment), `gas_pattern_service` (rule-based "which MQ is most elevated" hint), `yolo_service`, `ml_inference_service`, `hazard_service`, `dispatch_service`. Secrets live in `.env`.

7. Dashboard (React + Vite + Tailwind)

Page / component	What it shows
Dashboard	GaugeTile (custom SVG speedometer gauges), RiskScorePanel, CameraPanel (Live View polls 1 fps, stops after 60 s), SnapshotGallery (auto-refresh 5 s), DetectionTimeline
Alerts	Alert history with success/error per channel
Rooms	RoomProfile, appliances, EmergencyContacts
System	DeviceSetup: registered devices, room assignment, manual IP override
AI/ML, Analytics, Incidents, Reports	Placeholders ("Coming soon")

Run with `npm run dev` → `http://localhost:5173`; it talks to the backend on port 5000.

8. End-to-end scenarios

Scenario	What happens
Quiet night	1 Hz sampling, baseline slowly tracks, NORMAL, buzzer silent, OLED asleep, 30 s heartbeat.
Burnt toast	Gas rises → score ≥ 30 → PRECURSOR (short beep, snapshot, Telegram). Air clears → 15 s below 30 → NORMAL.
Smouldering fire, no flame	All three MQs rise (+ agreement bonus) → CONFIRMING (Telegram + Email with photo). With temperature also rising (BME280 fitted) → ≥ 65 for 8 s → FIRE_CONFIRMED → dispatch countdown.
Lighter flick / candle	2 s flame never satisfies the 8 s window → decays to NORMAL. A candle left burning reaches CONFIRMING but not FIRE_CONFIRMED without gas/temperature corroboration; the CNN labels it CONTROLLED_FLAME.
Owner on site, under control	Hold button 1.5 s → buzzer silenced, `ack_seq` +1, backend cancels dispatch.
WiFi/backend down	Detection, buzzer, OLED keep working; uploads dropped; WiFi retried every 10 s; re-registers on reconnect.
Sensor wire loose	I2C ping fails → `sensor_fault`, OLED FLT, that group ignored, auto-recovers.

9. Testing and verification

What	How	Result
Fusion, state machine, baseline freeze/clamp, window slope, flame debounce	Host-side C++ tests (g++)	9 scenarios pass (smoulder, flame only, flame + gas, transient, 60 °C floor, slow creep…)
Buzzer silence/re-arm, button debounce	Host-side C++ tests	13 checks pass
Firmware sensor-model code vs scikit-learn	`test_parity.py` compiles the real `sensor_ml.cpp`	678 windows: 100 % agreement, max probability diff 1e-4
Backend	`python -m unittest discover -s tests`	29 tests pass (hazard 9, dispatch 11, ingest 9)
Firmware build	Arduino esp32 core 3.3.12	Compiles, 46 % of the Huge APP partition
CNN	Held-out test set, FP32 vs INT8	See §5.1

> Test results above are as recorded in the project reference (29 Oct 2026). Full on-board testing (camera + buzzer together, vision timing, WiFi loss) was still pending at that date — update this row with whatever you have since tested on the real hardware.

10. Limitations and future work

1. **No real sensor training data yet** — sensor model is a stub; fusion weights and thresholds (35/25/30/10/10; 30/50/65) are hand-set.
2. **BME280 not fitted** — no temperature terms. Without flame, the maximum score is 30 + 10 + 10 = 50 (CONFIRMING); a fresh HAZARDOUS camera verdict can raise it to CONFIRMING but FIRE_CONFIRMED needs flame or temperature.
3. **MQ sensors** are not gas-specific, drift with age/humidity, need 24–48 h burn-in; MQ-7 is not heater-cycled (reduced CO sensitivity).
4. **CNN** trained on internet images, not yet validated on real OV3660 frames; 8 % of hazardous fires missed.
5. **YOLO** recall is limited and accuracy at the deployed 160 px is not measured.
6. Failed uploads are dropped, not buffered to SD.
7. Dispatch is simulated; the dashboard has no dispatch countdown/cancel panel yet; Twilio call escalation to a verified contact is planned.
8. Camera proxy needs the backend on the same LAN as the device (cloud hosting would need a tunnel).

11. Viva questions and answers

Concept and design

Q1. What is new compared with an ordinary smoke detector?

Multi-sensor fusion (gas ×3, UV, IR flame, optionally temperature) into one risk score; a staged state machine with persistence and hysteresis to cut false alarms; camera evidence classified on the device into controlled vs hazardous fire; remote alerts with photo; offline operation; a cancellable dispatch workflow.

Q2. Why fuse several sensors instead of using one?

Each sensor has its own false triggers: MQ sensors react to cooking and perfume, IR flame sensors to sunlight and lamps, UV to sunlight. A real fire produces several signals together, so agreement between independent sensors is much stronger evidence than any one sensor.

Q3. Why is the alarm logic rule-based and not purely ML?

It is a safety system: rules are transparent, deterministic, testable and need no fire data, which is hard and dangerous to collect. ML is layered on top and can only raise the risk, never suppress an alarm.

Q4. Why four states instead of fire / no fire?

Graded response. PRECURSOR gives an early, low-cost warning; CONFIRMING escalates alerts; FIRE_CONFIRMED triggers the strongest action (continuous tone, dispatch countdown). It also lets the system de-escalate gracefully.

Q5. How do you reject false alarms?

Flame debounce (4 of 10 polls), baseline-relative gas readings, multi-sensor agreement, an 8 s persistence window before FIRE, a 15 s cooldown, "flame alone cannot reach FIRE", and the CNN distinguishing a controlled flame from a hazardous fire.

Q6. What happens without internet?

All detection, the state machine, buzzer, OLED and on-device vision run locally on core 1 / the vision task. Only notifications and logging need the network, which is isolated in its own task on core 0.

Hardware

Q7. Why an external ADC (ADS1115)?

The camera uses most ADC1 pins, and ADC2 cannot be read while WiFi is active on the ESP32. The ADS1115 also gives 16-bit resolution over I2C.

Q8. Why the voltage dividers?

MQ modules are powered at 5 V and can output up to 5 V; the ADS1115 runs at 3.3 V and must not see more than its supply. A 10k/10k divider halves the signal; firmware multiplies by 2.

Q9. What does each MQ sensor detect?

MQ-2: smoke, LPG, combustible gases. MQ-7: carbon monoxide (incomplete combustion, smouldering). MQ-135: air quality, VOCs, smoke. They are broad-spectrum resistive sensors, not gas-specific, so we use relative change from baseline rather than ppm.

Q10. Why did you remove the battery and INA219?

The three MQ heaters draw about 450 mA, so the LiPo only lasted hours; the TP4056 had no power-path; and after the boost converter the INA219 could not report battery %. A 5 V / 2 A USB-C supply is simpler and more reliable for a fixed installation.

Q11. Why can't the native USB port be used?

GPIO20 is used as I2C SDA, and it is also native-USB D+. With "USB CDC On Boot" disabled, we flash and monitor via the UART USB port only.

Firmware

Q12. How do you use the two cores?

Core 1 runs `loop()` (sensing, decision, buzzer, OLED). Core 0 runs a FreeRTOS network task fed by a queue. A slow or failing HTTP request therefore never delays the alarm.

Q13. What is the baseline and why is it needed?

MQ readings vary by room, sensor unit, temperature and age. The device learns each room's normal level and measures deviation from it, which makes thresholds and ML features portable.

Q14. Couldn't a slow fire be learned as "normal"?

Learning only happens in NORMAL, freezes whenever any MQ is more than 25 % above baseline, uses a slow ~5 min EMA, and is clamped to 0.5–1.6× of its initial value.

Q15. What are the window features?

For the last 30 one-second samples: mean, standard deviation and least-squares slope per channel. Slope captures rate of rise; std captures flicker/turbulence. They feed the fusion rate terms and the sensor ML model.

Q16. How does the button cancel dispatch?

A 1.5 s long press increments `ack_seq` in the next upload. The backend sees the counter change and cancels the countdown. Using a counter (not a flag) makes it robust to dropped or repeated packets.

Machine learning

Q17. Why MobileNetV2 with α = 0.35 at 96 px?

It must fit in ESP32-S3 memory and run quickly. MobileNetV2 uses depthwise-separable convolutions; α = 0.35 scales channels to 35 %; 96×96 input keeps compute low. Transfer learning from ImageNet gives good accuracy with limited data.

Q18. What is INT8 quantization and what did it cost?

Weights and activations are mapped from 32-bit floats to 8-bit integers with a scale and zero-point, calibrated on 300 representative images. The model shrank from 1,573 KB to 612 KB (2.57×) and accuracy fell only from 97.76 % to 96.98 %; hazardous recall stayed ~92 %.

Q19. Which metric matters most and why?

Hazardous-fire recall (91.9 %), because missing a real fire is far worse than a false alarm. We also track the false-hazard alarm rate (1.4 %). A lower decision threshold (e.g. 0.20) raises recall to 96.4 % at 3.1 % false alarms.

Q20. How did you handle class imbalance?

Hazardous images are fewer (2,312 vs ~5,500–5,900 per other class in training), so class weights were used (≈ 2.97 for hazardous) and per-class recall and macro-F1 are reported instead of accuracy alone.

Q21. How did you avoid data leakage?

Exact/near duplicates (1,322) and cross-class conflicts (79) were removed, and the split is grouped so related images never appear in both train and test. For the sensor model, evaluation is leave-one-session-out because overlapping windows from the same recording are almost identical.

Q22. Why both a CNN and YOLO?

The CNN answers "what kind of scene is this?" (controlled vs hazardous). YOLO localizes flames, so we can measure flame area and its growth over time, which is a strong hazard cue (≥ 15 % of frame or ≥ 1.5× growth in 3 min).

Q23. Why is YOLO only a secondary signal?

Its recall is limited (validation recall 0.61 at 640 px; lower at the 160 px used on the board), so "no flame boxes" is never treated as "no fire". It can only add hazard evidence.

Q24. Why RandomForest + IsolationForest for sensors?

They work well on small tabular datasets, are fast, and trees convert directly into C arrays for a microcontroller. The IsolationForest is trained only on normal data, so it can flag unusual situations even for fire types we never recorded.

Q25. How do you know the C implementation matches Python?

`test_parity.py` compiles the actual firmware `sensor_ml.cpp` on the PC and compares it with scikit-learn on 678 windows: 100 % class and anomaly agreement, max probability difference 1e-4.

Q26. How does the model run on the ESP32?

TensorFlow Lite Micro. The `.tflite` files are read from the microSD into PSRAM; a tensor arena is allocated in PSRAM; the JPEG is decoded, resized with the same "area" interpolation used in training, shifted to int8, and invoked. A custom int8 fully-connected kernel was needed for YOLO's attention layer.

Q27. Is the sensor ML model trained on real data?

Not yet. The pipeline (collection, training, export, parity test) is complete and verified on synthetic data, but real labelled sessions still need to be recorded safely. Until then the device runs the rule-based fusion and the model is a stub.

Backend, alerts and safety

Q28. Why Flask and SQLite?

Lightweight, easy to run on a laptop, no separate database server; adequate for one site and a few devices. A larger deployment would move to PostgreSQL and a hosted server.

Q29. How do you avoid alert spam?

Alerts are sent only when the state changes, with a severity policy per state (Telegram for PRECURSOR; Telegram + Email for CONFIRMING and FIRE_CONFIRMED).

Q30. Why is the fire-department call simulated?

An automated, unverified emergency call risks false dispatches and is a legal liability. The workflow (countdown, cancel, escalation message) is fully implemented, but it sends to a simulation chat. A real system would use a certified monitoring service or call a verified contact (Twilio planned).

Q31. How does the device find the backend and vice versa?

The backend IP is in `config.h`. The device self-registers its id, IP and port every 60 s, so the backend always knows where to proxy camera requests; rooms are assigned on the dashboard without reflashing.

Q32. What security measures are there?

WiFi credentials in `secrets.h` and API tokens in `.env`, kept out of version control. The system runs on the local network. Not yet implemented: API authentication and HTTPS — listed as future work.

Results and reflection

Q33. What are your main results?

Working end-to-end architecture; firmware compiling for ESP32-S3 with on-device CNN + YOLO; INT8 CNN at 96.98 % accuracy and 91.9 % hazardous recall in 612 KB; 29 backend tests, host-side firmware tests and a 100 % parity test passing.

Q34. What was the hardest problem?

Fitting vision models on a microcontroller: memory (PSRAM placement, tensor arenas), matching pre-processing exactly between Python and C, and an unsupported operator in YOLO that needed a custom kernel. Also pin conflicts between camera, ADC and USB.

Q35. What would you do next?

Record real sensor sessions and train the sensor model; fit the BME280; validate the CNN on real OV3660 images and tune the threshold; measure YOLO at 160 px; add the dashboard dispatch panel; SD buffering for offline uploads; authentication/HTTPS; Twilio escalation.

12. Quick-reference numbers

Sampling	1 Hz fixed · flame poll 100 ms · window 30 samples
Thresholds	PRECURSOR 30 · CONFIRMING 50 · FIRE 65 · temperature floor 57 °C
Timing	Persistence 8 s · cooldown 15 s · FIRE hold 30 s · MQ warm-up 3 min · baseline ready ≈ 4 min · dispatch countdown 30 s
Uploads	30 s heartbeat · 1 s when elevated · immediate on state change · registration every 60 s
Fusion weights	Flame 35 · Temperature 25 · Gas 30 · Agreement 10 · UV 10
CNN (INT8)	612 KB · 96.98 % acc · hazardous recall 91.9 % · false hazard 1.4 % · test n = 2,944
YOLO11n	50 epochs, 640 px · val P 0.79 / R 0.61 / mAP50 0.69 · on-board INT8 160 px, 2.9 MB
I2C addresses	OLED 0x3C · ADS1115 0x48 · BME280 0x76
ADS channels	A0 MQ-2 · A1 MQ-7 · A2 MQ-135 · A3 UV
Pins	Flame 1 · button 2 · SDA 20 · SCL 21 · buzzer 42 · SD 38/39/40
Run	Backend `python run.py` (port 5000) · dashboard `npm run dev` (5173) · tests `python -m unittest discover -s tests`